



Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Richard Cajas
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad2 y honestidad.
Práctica Nro.	001
Tipo	Grupal
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Luis Darío Sinche C
Fecha	Viernes, 2 de Octubre del 2026
Horario	11h30 – 13h30
Lugar	Laboratorio
Tiempo planificado en el Sílabo	2 horas

2. Objetivo(s) de la Práctica

- Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio. Objetivos específicos: Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD). Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos). Comparar el acceso mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución. Identificar en el código ensamblador generado las instrucciones lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).



unl

Universidad
Nacional
de Loja

3. Materiales, Reactivos

- No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi: 1 Arduino UNO (ATmega328P a 16 MHz). 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω . 1 pulsador.

4. Equipos y Herramientas

- Computador con navegador web y conexión a internet.
- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos. FEIRNNR - Carrera de Computación Microchip Studio 7 (solo Windows), con el simulador del ATmega328P.
- Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology). Cuenta de GitHub y acceso al EVA para la entrega.

5. Procedimiento / Metodología

Trabajo en equipos. Regla general: desde la Parte B no se usan `pinMode()`, `digitalWrite()` ni `digitalRead()`; todo el control de pines se hace por registros. Duración total: 3 h.

Parte A. Reconocimiento del entorno y del microcontrolador

- A1. Ingrese a wokwi.com, cree un proyecto nuevo con Arduino Uno y guárdelo con el nombre APE1_<apellido1>_<apellido2>.
- A2. En la placa simulada, identifique el chip ATmega328P, el LED integrado "L" y el pin D13. Con la sección de Fundamento teórico, indique a qué puerto y bit corresponde D13.
- A3. Escriba el Código 1 (línea base con la API de Arduino), inicie la simulación y verifique que el LED "L" parpadea.

Código 1. Blink con la API de Arduino

```
// APE 1 - Código 1: Blink con la API de Arduino (línea base)
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
void setup() {
  pinMode(13, OUTPUT);      // D13 = PB5 (LED integrado "L")
}

void loop() {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

Parte B. El mismo comportamiento, ahora con registros

- B1. Reemplace el código por el Código 2 y verifique que el comportamiento es idéntico al del Código 1.

Código 2. Blink escribiendo registros

```
// APE 1 - Código 2: el mismo Blink, ahora escribiendo registros
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
// Regla: no se usa pinMode() ni digitalWrite()
void setup() {
  DDRB |= (1 << DDB5);      // PB5 (D13) como salida; los demás bits no cambian
}

void loop() {
  PORTB ^= (1 << DDB5);      // XOR: conmuta solo el bit 5
  delay(500);                // 500 ms encendido / 500 ms
                             // apagado
}
```

- B2. Sustituya $PORTB \wedge= (1 \ll PORTB5)$; por un par de líneas que enciendan ($|=$) y apaguen ($\& \sim$) el LED con un delay(500) entre ellas. Verifique.
- B3. Sustituya la conmutación por $PINB = (1 \ll PINB5)$; y verifique. Anote qué característica del ATmega328P lo permite.
- B4. Pruebe $PORTB = (1 \ll PORTB5)$; (asignación en lugar de OR). Funciona aquí, pero explique en el informe qué les pasaría a los otros bits de PORTB si hubiera más salidas conectadas en ese puerto. Lo comprobará en la Parte C.

Parte C. Circuito y laboratorio de bits

- C1. Arme en Wokwi el circuito de la Tabla 3. Puede hacerlo a mano o pegar el contenido del Anexo A en la pestaña diagram.json.

Tabla 3. Conexiones pin a pin

Componente	Pin del componente	Pin Arduino UNO	Puerto / bit	Configuración
LED0 (rojo) + R0 220 Ω	Ánodo (A) vía R0	D8	PB0	Salida
LED1 (amarillo) + R1 220 Ω	Ánodo (A) vía R1	D9	PB1	Salida
LED2 (verde) + R2 220 Ω	Ánodo (A) vía R2	D10	PB2	Salida
LED3 (azul) + R3 220 Ω	Ánodo (A) vía R3	D11	PB3	Salida
LED0 ... LED3	Cátodo (C)	GND	—	—
Pulsador	1.I	D2	PD2	Entrada con pull-up interna
Pulsador	2.I	GND	—	—
LED integrado "L"	—	D13	PB5	Salida (latido)

Nota sobre pines: no use D0 ni D1, porque son la UART que usa el Monitor Serie. El pulsador va a GND porque se usa la resistencia pull-up interna, así que se lee 1 suelto y 0 presionado (activo en bajo). En el diagram.json el rebote del pulsador está desactivado ("bounce": "0"); en hardware real habrá que tratar el rebote.

- C2. Cargue el Código 3, inicie la simulación y abra el Monitor Serie (9600 baudios).

Código 3. Laboratorio de bits (Arduino UNO en Wokwi, sin bibliotecas externas)

```

/*
  APE 1 - Código 3: Laboratorio de bits en los puertos B y D
  Placa : Arduino UNO (ATmega328P @ 16 MHz) - simulacion en
  Wokwi Libs : ninguna (avr/io.h llega incluido por Arduino.h)
  Pines : PB0..PB3 (D8..D11) LEDs | PD2 (D2) pulsador a GND | PB5 (D13)
  LED L
  Regla : NO se usa pinMode(), digitalWrite() ni digitalRead()
*/
const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3 const
uint8_t NUM_MODOS = 3;
const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
const unsigned long PERIODO_MS = 300;

uint8_t modo = 0;
uint8_t patron =
PATRON_INICIAL[0]; uint8_t
btnAnterior = 1; // con pull-up, en reposo se lee 1
unsigned long tAnterior = 0;
void imprimirBin8(const char *etiqueta, uint8_t v) {
  Serial.print(etiqueta);
  for (int8_t i = 7; i >= 0; i--) {
    Serial.print((v >> i) & 1); // extrae el bit i
  }
  Serial.println();
}

void escribirLeds(uint8_t valor) {
  // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se
  conservan PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
}

void demoOperaciones() {
  uint8_t a = 0b11001100;
  uint8_t b = 0b10101010;
  imprimirBin8("a = ", a);
  imprimirBin8("b = ", b);
  imprimirBin8("a & b = ", a & b);
  imprimirBin8("a | b = ", a | b);
  imprimirBin8("a ^ b = ", a ^ b);
  imprimirBin8("~a = ", (~a));
}

```

```

imprimirBin8("a >> 2 = ", a >> 2);
Serial.print("~a sin cast, en BIN:
"); Serial.println(~a, BIN);
} // ver pregunta de control
6

void setup() {
  DDRB |= MASK_LEDS | (1 << DDB5); // salidas: PB0..PB3 y PB5
  DDRD &= ~(1 << DDD2); // PD2 entrada (bit DDR = 0)
  PORTD |= (1 << PORTD2); // pull-up interna (bit PORT = 1)

  Serial.begin(9600);
  Serial.println(F("APE 1 - Laboratorio de bits"));
  demoOperaciones();
  escribirLeds(patron);
}

void loop() {
  // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
  uint8_t btn = (PIND >> PIND2) & 1; // 1 = suelto, 0 = presionado if
  (btnAnterior == 1 && btn == 0) { // flanco de bajada
    modo = (modo + 1) % NUM_MODOS;
    patron = PATRON_INICIAL[modo];
    Serial.print(F("Modo: "));
    Serial.println(modo);
  }
  btnAnterior = btn;

  // 2) Actualizacion periodica sin bloquear el
  programa if (millis() - tAnterior >= PERIODO_MS) {
    tAnterior = millis();
    switch (modo) {
      case 0: // contador binario de 4
        patron = (patron + 1) & MASK_LEDS; bits
        break;
      case 1: // rotacion circular a la izquierda patron = ((patron << 1) |
        (patron >> 3)) & MASK_LEDS;
        break;
      case 2: // alternancia con XOR
        patron ^= MASK_LEDS;
        break;
    }
    escribirLeds(patron);
    PINB = (1 << PINB5); // escribir 1 en PINB5 conmuta PB5
    imprimirBin8("PORTB = ", PORTB);
  }
}

```

- C3. Antes de mirar el Monitor Serie, calcule a mano los resultados de demoOperaciones() y luego compárelos con la salida.
- C4. Presione el pulsador y describa los tres modos. Para cada modo, explique qué operación a nivel de bit produce el patrón.
- C5. Observe la línea PORTB = ... del Monitor Serie e identifique qué bits corresponden a los LEDs y cuál al LED "L".
- C6. Cambie escribirLeds() por PORTB = valor; y describa qué ocurre con el LED "L". Relacione el resultado con el paso B4 y restaure la versión original.
- C7. Complete la Tabla 4 en el informe.

Tabla 4. Configuración de registros (para completar)

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido			—
D11 (PB3)	LED3 apagado			—
D2 (PD2)	Pulsador suelto			
D2 (PD2)	Pulsador presionado			
D13 (PB5)	LED "L" encendido			—

Parte D. Costo de la API frente al acceso a registros

- D1. En un proyecto nuevo de Wokwi (Arduino Uno, sin componentes adicionales), cargue el Código 4 y ejecútelo tres veces.

Código 4. Medición de tiempo de escritura

```
// APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
const uint16_t N = 10000;

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);

  unsigned long t0 = micros();
  for (uint16_t i = 0; i < N; i++) {
    digitalWrite(13, HIGH);
    digitalWrite(13, LOW);
  }
  unsigned long t1 = micros();
  for (uint16_t i = 0; i < N; i++) {
    PORTB |= (1 << PORTB5);
    PORTB &= ~(1 << PORTB5);
  }
  unsigned long t2 = micros();

  Serial.print(F("digitalWrite : "));
  Serial.print((t1 - t0) / (2.0 * N),
  3); Serial.println(F(" us por
  escritura"));
  Serial.print(F("Registro : "));
  Serial.print((t2 - t1) / (2.0 * N),
  3); Serial.println(F(" us por
  escritura"));
}
```

```
void loop() {}
```

- D2. Registre los tiempos de las tres ejecuciones, calcule el promedio y la relación digitalWrite/registro. Los tiempos incluyen la sobrecarga del bucle y de la interrupción del Timer0, por lo que interesa la comparación, no el valor absoluto.

Parte E. Del C al set de instrucciones con Microchip Studio

- E1. En Microchip Studio: File › New › Project › GCC C Executable Project, dispositivo ATmega328P.
- E2. Reemplace el contenido de main.c por el Código 5 y compile (Build › Build Solution, F7). Verifique que no haya errores.

Código 5. C puro para Microchip Studio (sin Arduino)

```
/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler
 * (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
  DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como
  salidas DDRD &= ~(1 << DDD2); // PD2 como entrada
  PORTD |= (1 << PORTD2); // pull-up interna en PD2

  while (1)
  {
    if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
      PORTB &= ~(1 << // apaga PB5
      PORTB5); // pulsador presionado (lee
      0)
    } else {
      PORTB |= (1 << PORTB5); // enciende PB5
    }
    PORTB ^= (1 << PORTB0); // conmuta PB0 en cada
    vuelta
  }
}
```

E3. Abra el archivo .iss (Solution Explorer > Output Files). Localice la función main y complete la Tabla 5 con las instrucciones generadas para cada línea de C y sus ciclos según el AVR Instruction Set Manual.

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);		
DDRD &= ~(1 << DDD2);		
PORTD = (1 << PORTD2);		
if (PIND & (1 << PIND2))		
PORTB &= ~(1 << PORTB5);		
PORTB = (1 << PORTB5);		
PORTB ^= (1 << PORTB0);		

E4. Seleccione el simulador (Project > Properties > Tool > Simulator) e inicie la depuración con Debug > Start Debugging and Break (Alt+F5).

E5. Abra Debug > Windows > I/O, seleccione PORTB y PORTD y avance instrucción por instrucción (F10/F11). Observe cómo cambian DDRB, PORTB y PORTD. Simule el pulsador modificando el bit 2 de PIND en la vista I/O y observe el efecto en PB5. Si el simulador no refleja la pull-up, ponga PIND2 en 1 a mano para representar “suelto”.

E6. En Debug > Windows > Processor Status, use el contador de ciclos para medir cuántos ciclos tarda una vuelta del while(1) con el pulsador suelto y con el pulsador presionado.

Alternativa sin Windows: en <https://godbolt.org> seleccione el lenguaje C y un compilador “AVR gcc” con las opciones -O1 -mmcu=atmega328p, pegue el Código 5 y complete la Tabla 5 con el ensamblador mostrado. Los pasos E4 a E6 se reemplazan por el análisis manual de ciclos. Las instrucciones pueden variar ligeramente según la versión del compilador y el nivel de optimización.

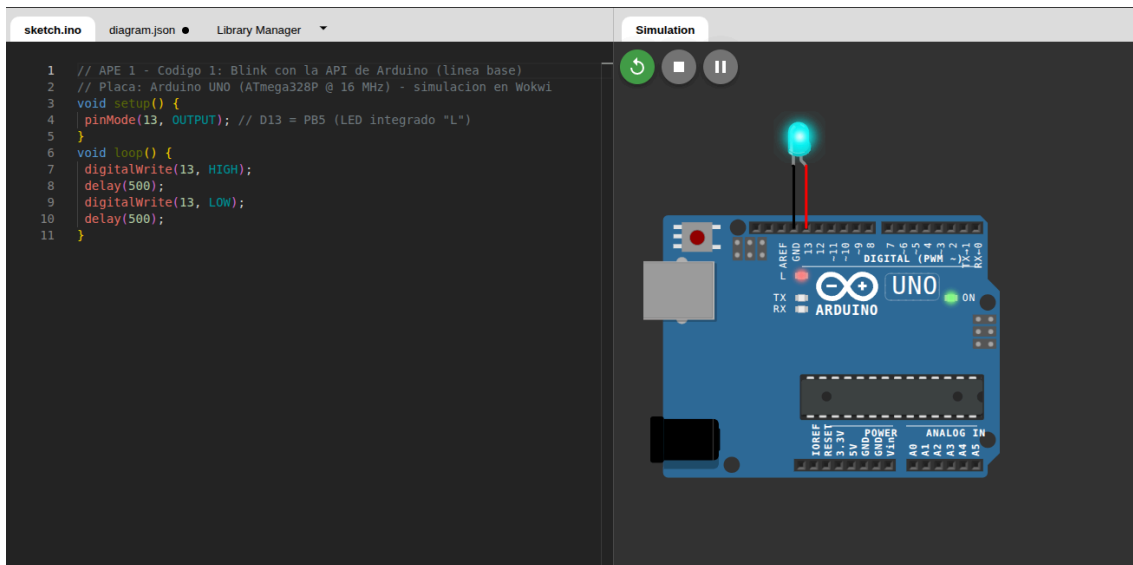
Parte F. Cierre y entrega

F1. Publique los proyectos de Wokwi como públicos y copie sus enlaces.

F2. Suba los códigos, el diagram.json y el informe al repositorio de GitHub de la pareja y entregue el informe en el EVA.

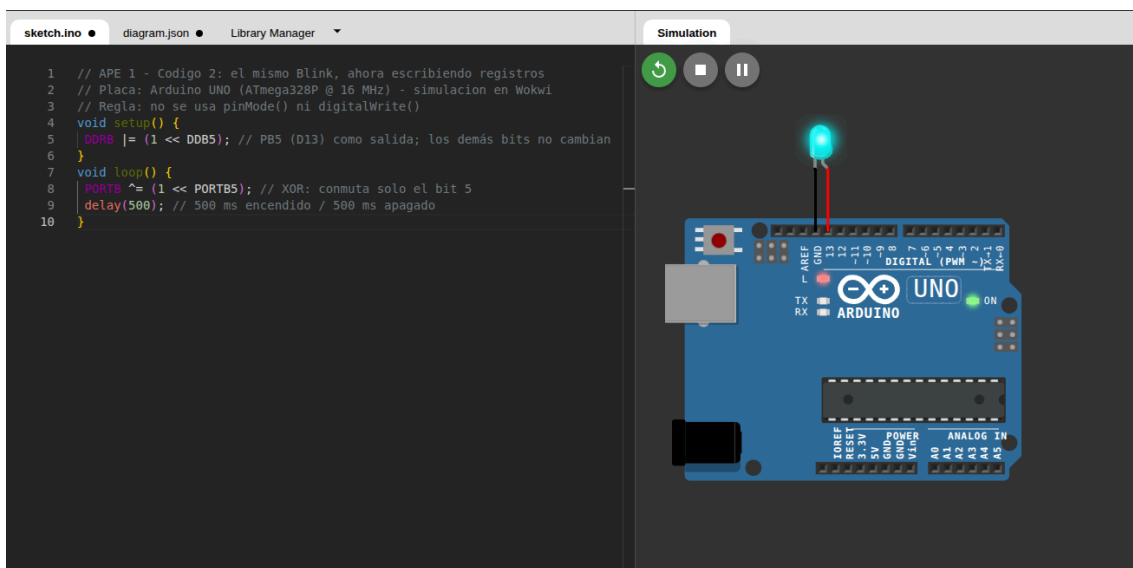
6. Resultados

A3:



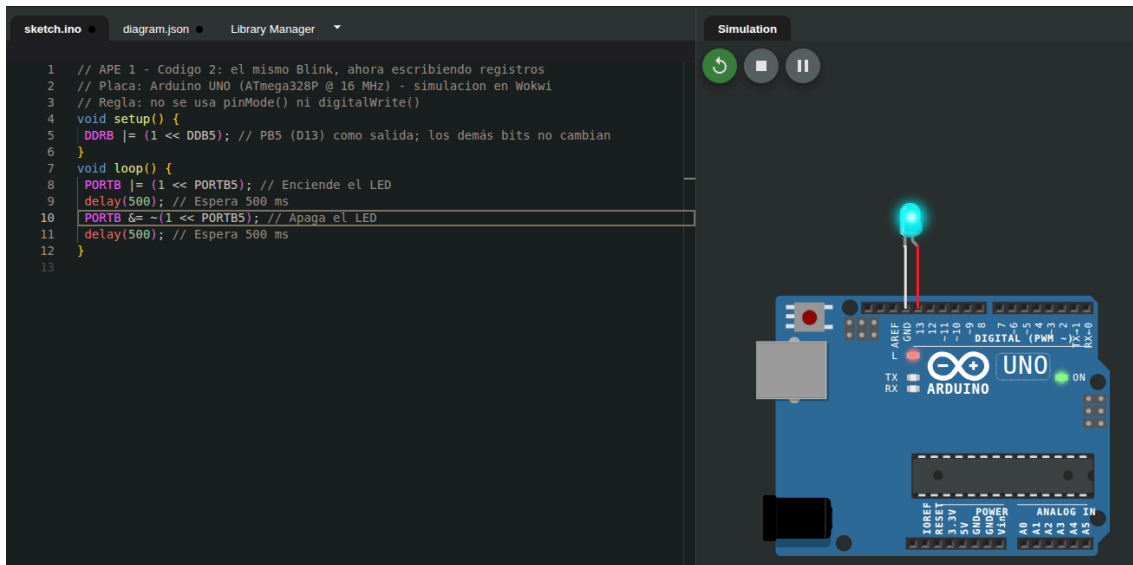
<https://wokwi.com/projects/477104112164360193>

B1:



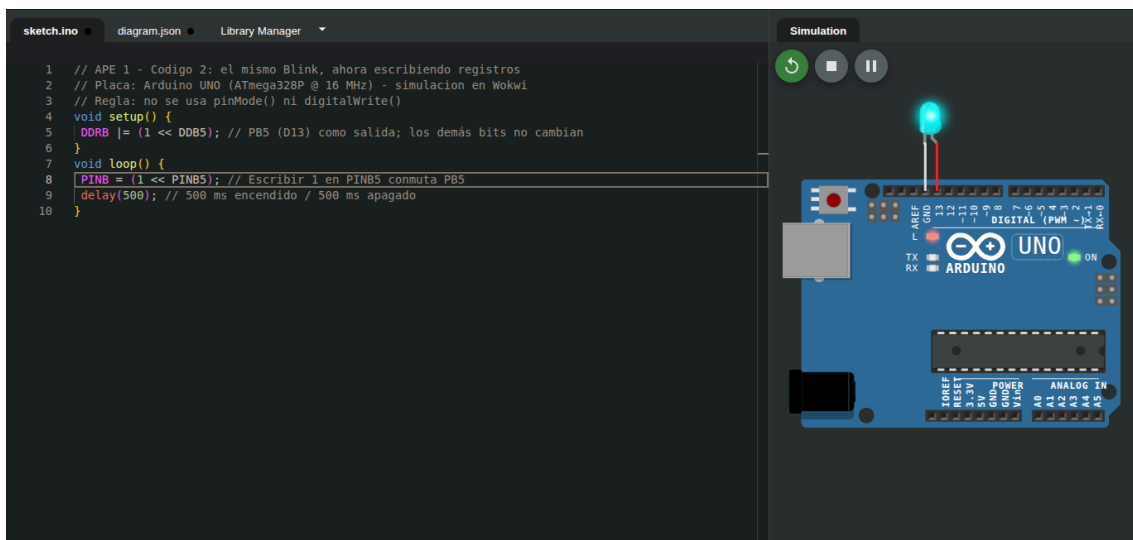
<https://wokwi.com/projects/477104661186429953>

B2:



<https://wokwi.com/projects/477104980086840321>

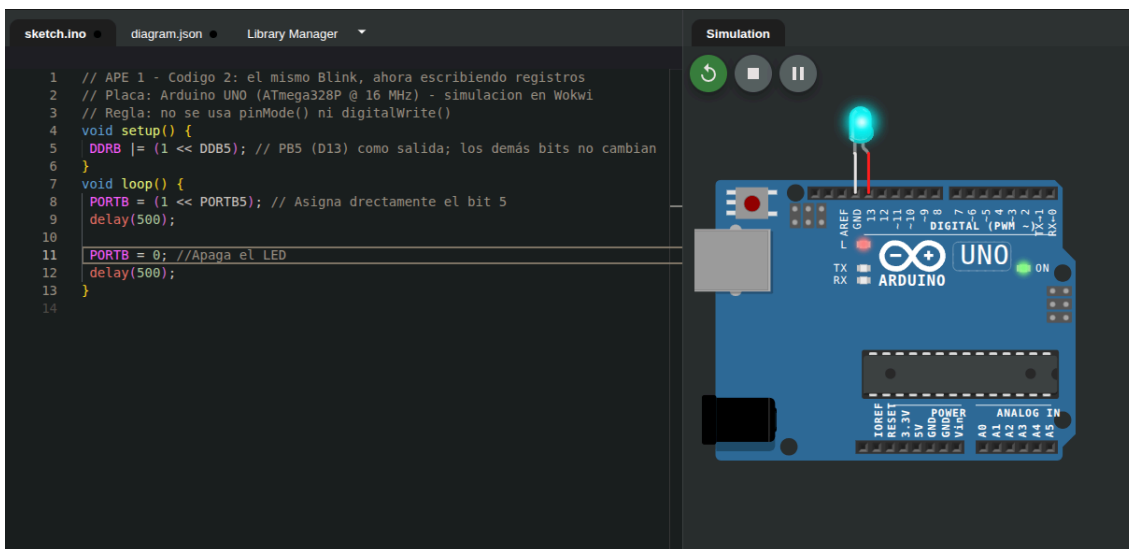
B3:



<https://wokwi.com/projects/477105208770878465>

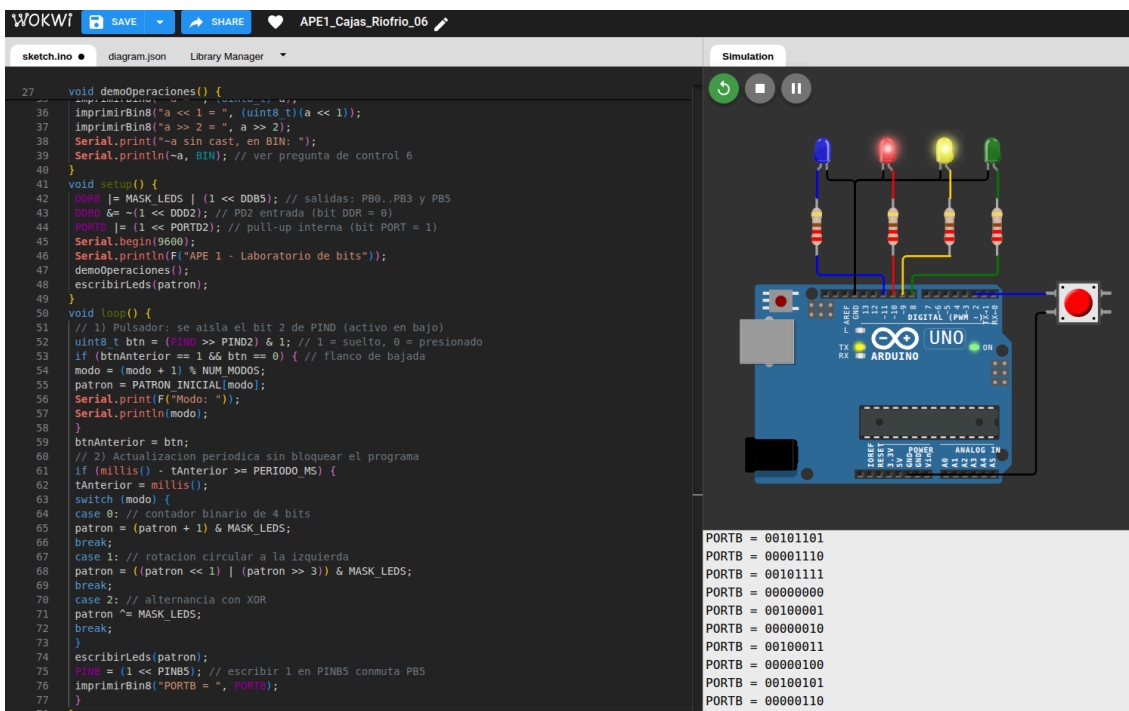
La característica del ATmega328p que lo permite es que escribir un 1 en un bit de PINx conmuta el bit correspondiente de PORTx.

B4:



<https://wokwi.com/projects/477105477636402177>

C2:

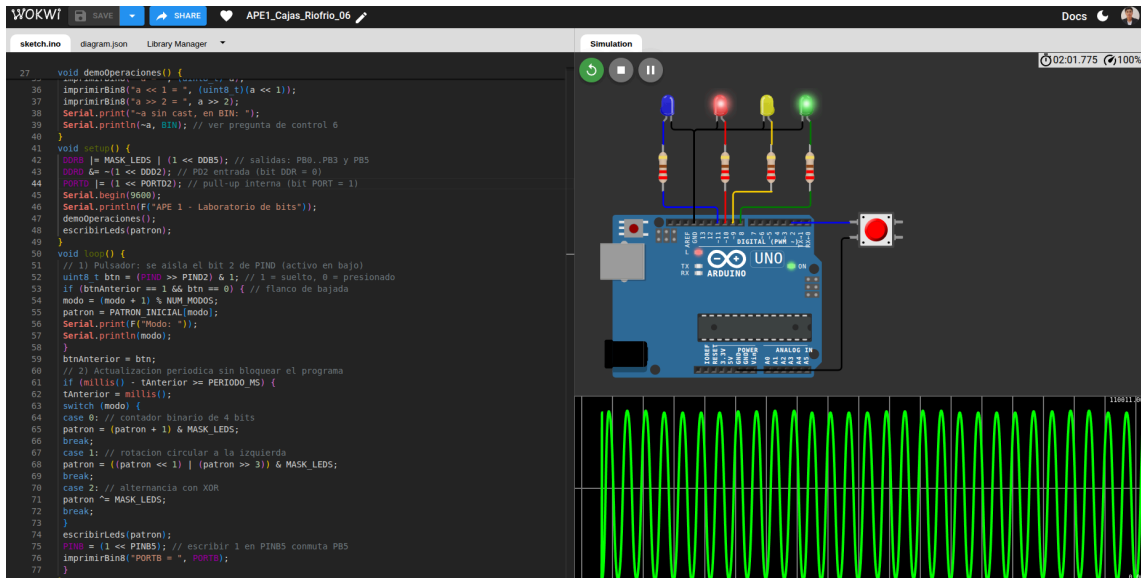




unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación



<https://wokwi.com/projects/477106488953820161>

C3

Tenemos

$a = 11001100$

$b = 10101010$

① AND $a \& b$

$$\begin{array}{r} 11001100 \\ \& 10101010 \\ \hline 10001000 \end{array}$$

② OR $a | b$

$$\begin{array}{r} 11001100 \\ | 10101010 \\ \hline 11101110 \end{array}$$

③ XOR $a \wedge b$

$$\begin{array}{r} 11001100 \\ \wedge 10101010 \\ \hline 01100110 \end{array}$$

④ Not $\sim a$

$a = 11001100$

$\sim a = 00110011 //$

⑤ Desplazamiento $a \ll 1$

$$\begin{array}{r} 11001100 \ll 1 \\ 11001100 \\ 10011000 // \end{array}$$

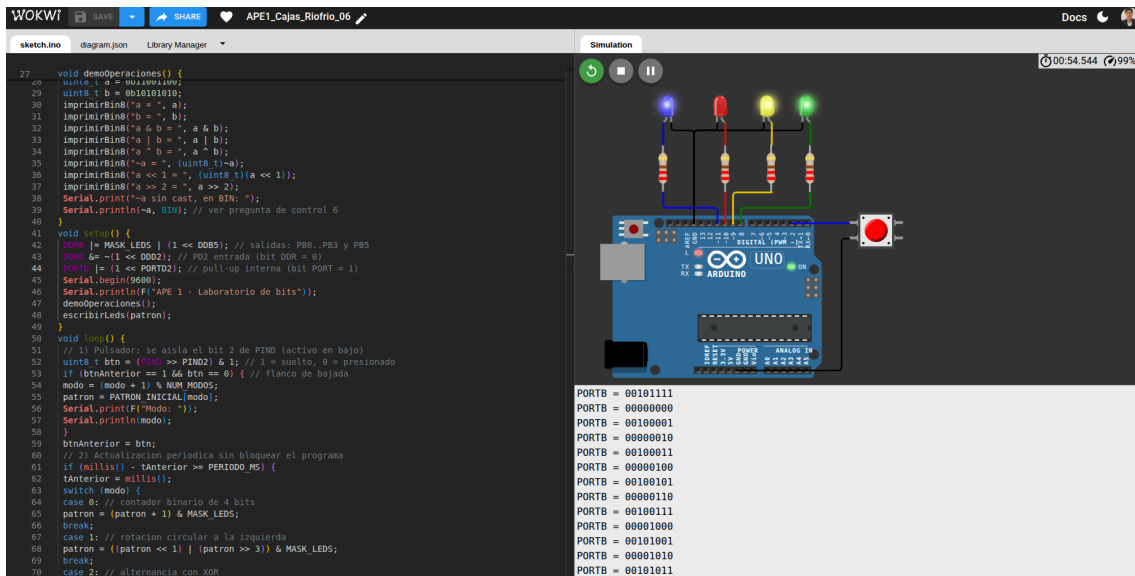
⑥ Desplazamiento $a \gg 2$

$$\begin{array}{r} 11001100 \gg 2 \\ 11001100 \gg 1 = 01100110 \\ 01100110 \gg 1 = \boxed{00110011} // \end{array}$$



C4:

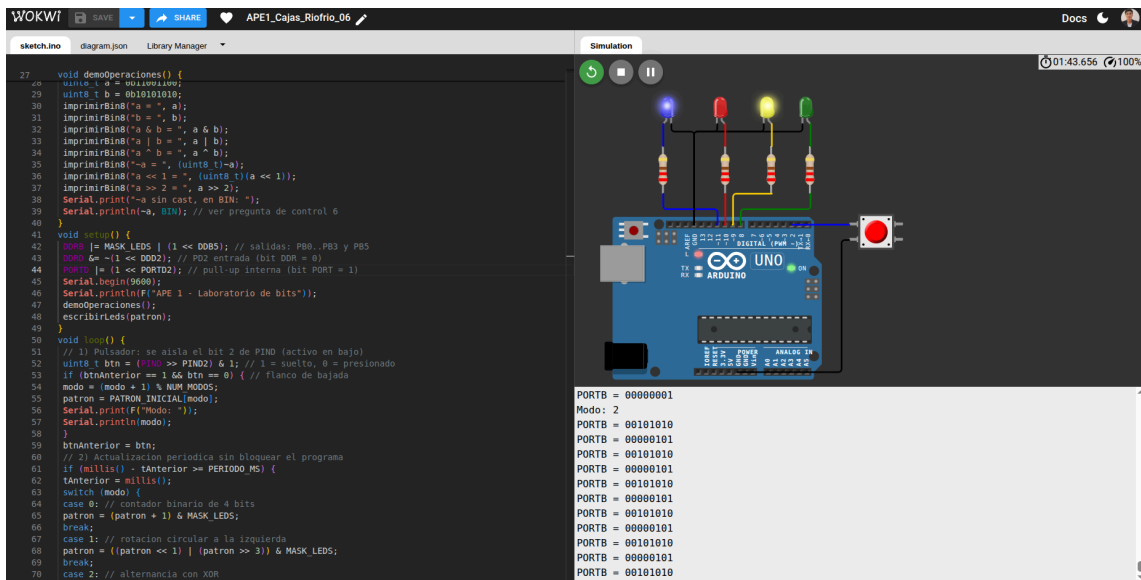
MODO 1: En este modo el LED verde se mantiene parpadeando constantemente mientras que los demás van parpadeando uno tras otro de derecha a izquierda.



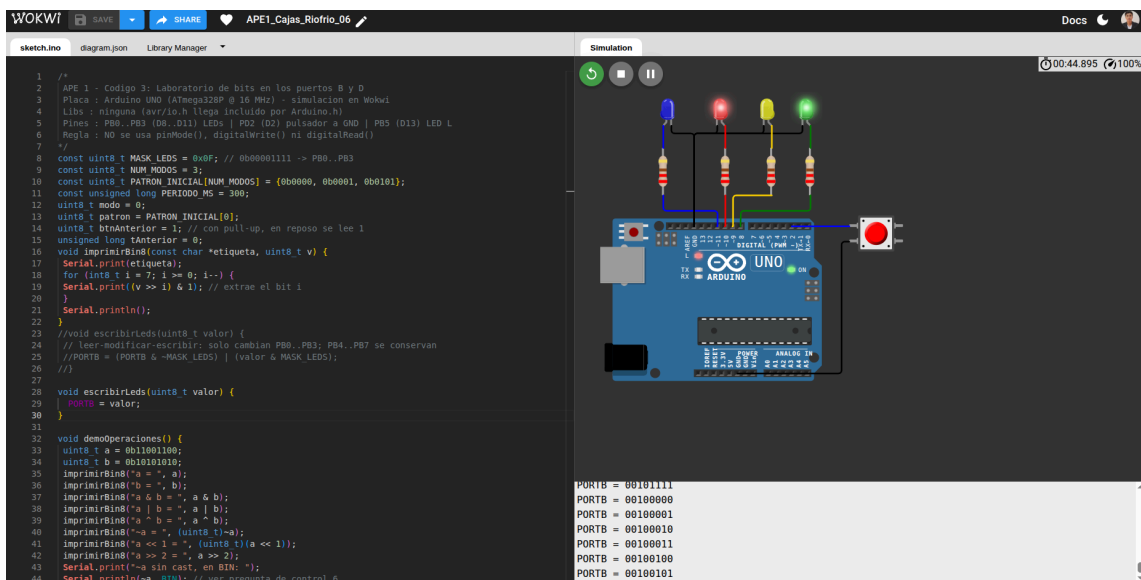
MODO 2: En este modo cada led sigue una secuencia de parpadeo de derecha a izquierda.



MODO 3: La secuencia es que se encienden en pares, separados por un led, por ejemplo enciende el azul y el amarillo y se mantiene apagado el rojo y el verde, y viceversa, luego encienden todos y se repite la secuencia.



C6:



Al sustituir la función escribirLeds() por PORTB = valor;, el LED “L” de D13 (PB5) deja de parpadear de manera adecuada. Esto sucede porque la asignación directa altera todos los bits del registro PORTB, incluyendo PB5, aunque el valor solo tiene el patrón dirigido a PB0–PB3. Por otro lado, la versión original emplea una operación de lectura-modificación-escritura que posibilita modificar solo PB0–PB3 y mantener el estado de los otros bits. Esto está vinculado con B4, donde se notó que PORTB = (1 << PORTB5) puede alterar las demás salidas del puerto.

C7:

PIN	FUNCION	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED 0	1	1	—



	encendido			
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED "L" encendido	1	1	—

D1:

1ra ejecucion:

WOKWI

SAVE

SHARE

♥ APE1_Cajas_Riofrio_07

sketch.ino diagram.json Library Manager

```
1 // APE 1 - Codigo 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
3 const uint16_t N = 10000;
4 void setup() {
5   Serial.begin(9600);
6   pinMode(13, OUTPUT);
7   unsigned long t0 = micros();
8   for (uint16_t i = 0; i < N; i++) {
9     digitalWrite(13, HIGH);
10    digitalWrite(13, LOW);
11  }
12  unsigned long t1 = micros();
13  for (uint16_t i = 0; i < N; i++) {
14    PORTB |= (1 << PORTB5);
15    PORTB &= ~(1 << PORTB5);
16  }
17  unsigned long t2 = micros();
18  Serial.print(F("digitalWrite : "));
19  Serial.print((t1 - t0) / (2.0 * N), 3);
20  Serial.println(F(" us por escritura"));
21  Serial.print(F("Registro : "));
22  Serial.print((t2 - t1) / (2.0 * N), 3);
23  Serial.println(F(" us por escritura"));
24 }
25 void loop() {}
26
```

Simulation

digitalWrite : 3.395 us por escritura

Registro : 0.220 us por escritura

2da ejecucion



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

WOKWI SAVE SHARE APE1_Cajas_Riofrio_07

sketch.ino diagram.json Library Manager

```
1 // APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 const uint16_t N = 10000;
4 void setup() {
5   Serial.begin(9600);
6   pinMode(13, OUTPUT);
7   unsigned long t0 = micros();
8   for (uint16_t i = 0; i < N; i++) {
9     digitalWrite(13, HIGH);
10    digitalWrite(13, LOW);
11  }
12  unsigned long t1 = micros();
13  for (uint16_t i = 0; i < N; i++) {
14    PORTB |= (1 << PORTB5);
15    PORTB &= ~(1 << PORTB5);
16  }
17  unsigned long t2 = micros();
18  Serial.print(F("digitalWrite : "));
19  Serial.print((t1 - t0) / (2.0 * N), 3);
20  Serial.println(F(" us por escritura"));
21  Serial.print(F("Registro : "));
22  Serial.print((t2 - t1) / (2.0 * N), 3);
23  Serial.println(F(" us por escritura"));
24 }
25 void loop() {}
26
```

Simulation

digitalWrite : 3.395 us por escritura
Registro : 0.220 us por escritura

3ra ejecucion

WOKWI SAVE SHARE APE1_Cajas_Riofrio_07

sketch.ino diagram.json Library Manager

```
1 // APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
2 // Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulación en Wokwi
3 const uint16_t N = 10000;
4 void setup() {
5   Serial.begin(9600);
6   pinMode(13, OUTPUT);
7   unsigned long t0 = micros();
8   for (uint16_t i = 0; i < N; i++) {
9     digitalWrite(13, HIGH);
10    digitalWrite(13, LOW);
11  }
12  unsigned long t1 = micros();
13  for (uint16_t i = 0; i < N; i++) {
14    PORTB |= (1 << PORTB5);
15    PORTB &= ~(1 << PORTB5);
16  }
17  unsigned long t2 = micros();
18  Serial.print(F("digitalWrite : "));
19  Serial.print((t1 - t0) / (2.0 * N), 3);
20  Serial.println(F(" us por escritura"));
21  Serial.print(F("Registro : "));
22  Serial.print((t2 - t1) / (2.0 * N), 3);
23  Serial.println(F(" us por escritura"));
24 }
25 void loop() {}
26
```

Simulation

digitalWrite : 3.395 us por escritura
Registro : 0.220 us por escritura

<https://wokwi.com/projects/477107763590974465>

Promedio final 3.395



Tabla 4. Configuración de registros (para completar)

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	0	—
D11 (PB3)	LED3 apagado	1	0	—
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED "L" encendido	1	1	—

D2:

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in r24,0x4 ori r24,lo8(33) out 0x4,r24	3 (1 por instrucción)
DDRD &= ~(1 << DDD2);	cbi 0xa,2	2
PORTD = (1 << PORTD2);	sbi 0xb,2	2
if (PIND & (1 << PIND2))	sbis 0x9,2	1 o 2
PORTB &= ~(1 << PORTB5);	cbi 0x5,5	2
PORTB = (1 << PORTB5);	sbi 0x5,5	2
PORTB ^= (1 << PORTB0);	in r24,0x eor r24,r25 out 0x5,r24	3 (1 por instrucción)

E2:

```
/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 * Created: 6/10/2026 1:46:29
 * Author : Richard Gajon
 */
#include <avr/io.h>
int main(void)
{
    DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
    DDRD &= ~(1 << DDD2); // PD2 como entrada
    PORTD |= (1 << PORTD2); // pull-up interna en PD2
    while (1)
    {
        if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
            PORTB &= ~(1 << PORTB5); // apaga PB5
        } else { // pulsador presionado (lee 0)
            PORTB |= (1 << PORTB5); // enciende PB5
        }
    }
}
```

Output

```
Build
C:\Program Files (x86)\Atmel\Studio7.0\toolchain\avr8\avr8-gnu-toolchain\bin\avr-size.exe "GccApplication1.elf"
text data bss dec hex filename
178 0 0 178 b2 GccApplication1.elf
Done executing task "RunCompilerTask".
Task "RunOutputFileVerifyTask"
Program Memory Usage : 178 bytes 0,5 % Full
Data Memory Usage : 0 bytes 0,0 % Full
Warning: Memory Usage estimation may not be accurate if there are sections other than .text sections in ELF file
Done executing task "RunOutputFileVerifyTask".
Done building target "CoreBuild" in project "GccApplication1.cproj".
Target "PostBuildEvent" skipped, due to false condition; ('$(PostBuildEvent)' != '') was evaluated as ('' != '').
Target "Build" in file "C:\Program Files (x86)\Atmel\Studio7.0\VsAvr.common.targets" from project "c:\users\usuario itc\Documents\Atmel Studio7.0\GccApplication1\GccApplication1.cproj" (entry point):
Done building target "Build" in project "GccApplication1.cproj".
Done building project "GccApplication1.cproj".

Build succeeded.
----- Build: 1 succeeded or up-to-date, 0 failed, 0 skipped -----
```



El código se ejecutó y compiló sin ningún error

E3:

```
7c: 0c 94 00 00    jmp 0    ; 0x0 <__vectors>

00000080 <main>:
* Author : Richard Cajas
*/
#include <avr/io.h>
int main(void)
{
    DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
80: 84 b1          in  r24, 0x04    ; 4
82: 81 62          ori r24, 0x21    ; 33
84: 84 b9          out 0x04, r24    ; 4

    DDRD &= ~(1 << DDD2); // PD2 como entrada
86: 8a b1          in  r24, 0x0a    ; 10
88: 8b 7f          andi r24, 0xfB    ; 251
8a: 8a b9          out 0x0a, r24    ; 10

    PORTD |= (1 << PORTD2); // pull-up interna en PD2
8c: 8b b1          in  r24, 0x0b    ; 11
8e: 84 60          ori r24, 0x04    ; 4
90: 8b b9          out 0x0b, r24    ; 11

    while (1)
    {
        if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
92: 4a 9b          sbis 0x09, 2 ; 9
94: 04 c0          rjmp .+8        ; 0x9e <main+0x1e>
                                // apaga PB5
        PORTB &= ~(1 << PORTB5);
96: 85 b1          in  r24, 0x05    ; 5
98: 8f 7d          andi r24, 0xDF    ; 223
9a: 85 b9          out 0x05, r24    ; 5
9c: 03 c0          rjmp .+6        ; 0xa4 <main+0x24>
        } else { // pulsador presionado (lee 0)
                                // enciende PB5
        PORTB |= (1 << PORTB5);
9e: 85 b1          in  r24, 0x05    ; 5
a0: 80 62          ori r24, 0x20    ; 32
a2: 85 b9          out 0x05, r24    ; 5
        }

        PORTB ^= (1 << PORTB0); // conmuta PB0 en cada vuelta
a4: 95 b1          in  r25, 0x05    ; 5
a6: 81 e0          ldi r24, 0x01    ; 1
    }
```

Tabla 5. Correspondencia C – ensamblador (para completar)

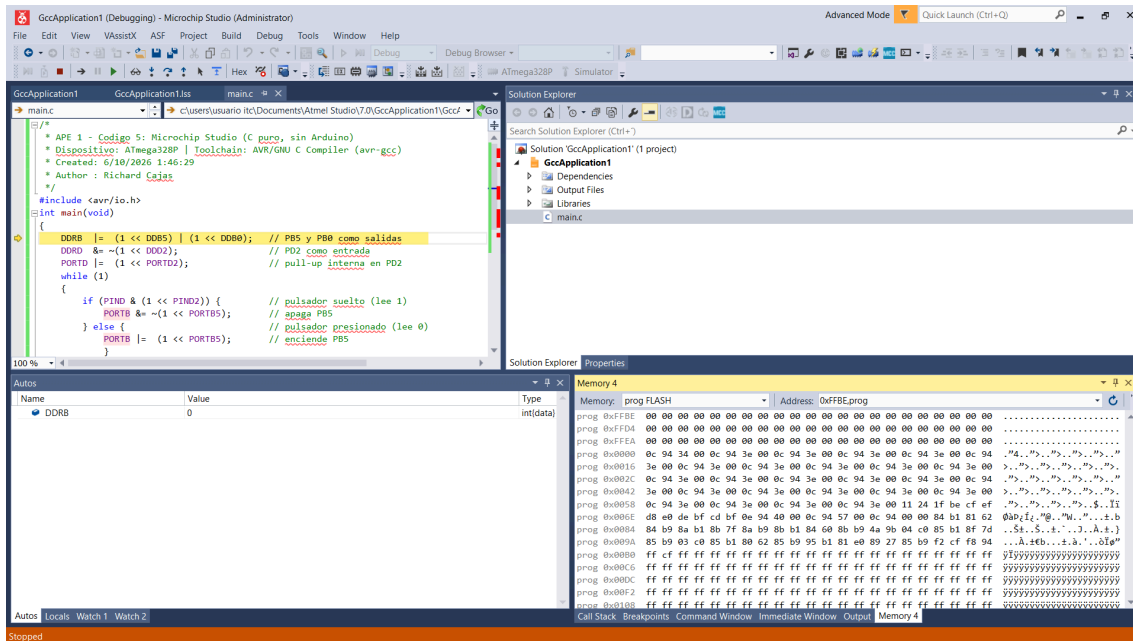
Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in, ori, out	3
DDRD &= ~(1 << DDD2);	in, andi, out	3
PORTD = (1 << PORTD2);	in, ori, out	3
if (PIND & (1 << PIND2))	sbis, rjmp	2
PORTB &= ~(1 << PORTB5);	in, andi, out, rjmp	4
PORTB = (1 << PORTB5);	in, ori, out	3
PORTB ^= (1 << PORTB0);	in, ldi, eor, out	4

E4:

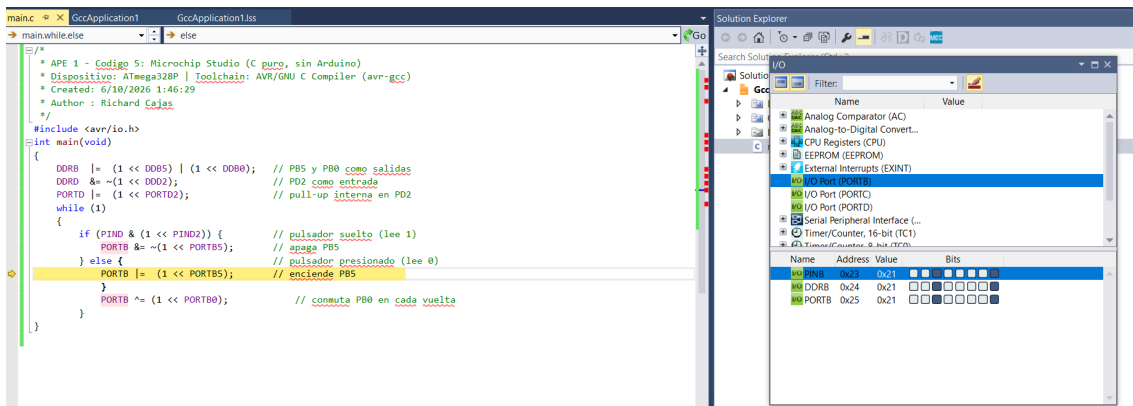


Universidad
Nacional
de Loja

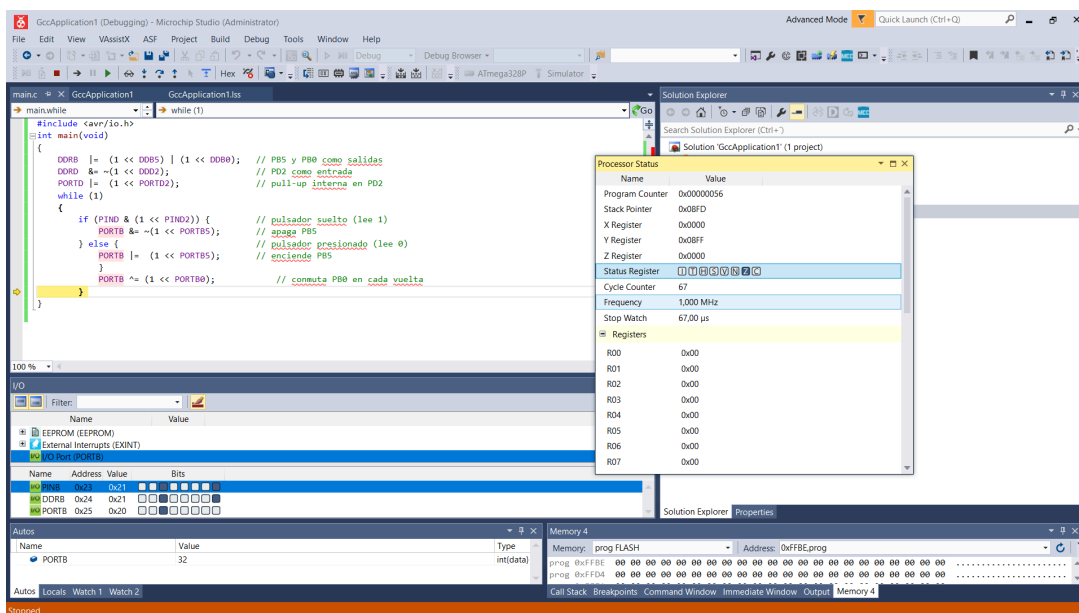
FEIRNNR - Carrera de Computación



E5:



E6:





7. Preguntas de Control

1. ¿Qué diferencia a un microcontrolador de un microprocesador?
Enumere los bloques del ATmega328P que identificó en la práctica.
Un microprocesador como el de mi compu contiene solo la unidad de procesamiento (CPU) y necesita que le conectes memoria y periféricos externos para funcionar. Un microcontrolador como el ATmega328P es una "computadora en un solo chip": ya integra la CPU, la memoria (RAM y Flash) y los puertos de entrada/salida (I/O) en la misma pastilla.
2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard?
Relacione su respuesta con el uso de la macro F("...") en el Código
Implica que el microcontrolador tiene memorias y buses de comunicación físicamente separados para los Datos (RAM) y para las Instrucciones/Programa (Flash). Esto le permite a la CPU leer una instrucción y un dato al mismo tiempo, haciéndolo más rápido y por lo que la memoria RAM del ATmega328P es pequeñísima (apenas 2 KB). Por defecto, al imprimir texto normal, el sistema copia ese texto de la Flash a la RAM, desperdiciando espacio. La macro F() aprovecha la separación de la arquitectura Harvard para decirle al sistema: "deja este texto guardado en la memoria Flash y léelo desde ahí", ahorrando RAM valiosa.
3. Explique por qué `PORTB = (1 << PORTB5);` puede alterar otras salidas y `PORTB |= (1 << PORTB5);` no, usando lo observado en los pasos B4 y C6.
La asignación directa (`PORTB = valor`) altera todos los bits del registro PORTB al mismo tiempo, modificando pines que no querías cambiar. Por otro lado, usar el operador `|=` realiza una operación de lectura-modificación-escritura que permite cambiar solo el bit deseado (como PB5) y mantener el estado actual de los demás bits intacto.
4. Para usar PD2 como entrada con pull-up, ¿qué valores deben tener `DDRD2` y `PORTD2`? ¿Qué se lee en `PIND2` con el pulsador suelto y presionado, y por qué?
Según tu Tabla 4 y la sección C7, el bit `DDRx` debe ser 0 y el bit `PORTx` debe ser 1. Cuando el pulsador está suelto, el bit `PINx` leído es 1. Cuando el pulsador está presionado, el bit `PINx` leído es 0.
5. Calcule a mano `0b11001100 & 0b10101010`, `|` y `^`, y verifique sus resultados con el Monitor Serie.
Me dieron los mismos resultados al calcular a mano y en el monitor serie.
6. Si `a` es de tipo `uint8_t`, ¿por qué `Serial.println(~a, BIN)` imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?
Como se observa en la consola de tu simulación (Modo 1), imprimir `~a` sin "cast" provoca que se muestren más bits (se imprime



- Durante esta práctica comprobé que manipular directamente los registros (DDR, PORT, PIN) del ATmega328P es drásticamente más rápido que usar la API nativa de Arduino. Medir los tiempos de ejecución me demostró que el acceso a registros toma apenas 0.220 microsegundos por escritura, frente a los 3.395 microsegundos de digitalWrite, un rendimiento que será clave cuando necesite programar aplicaciones donde el tiempo de respuesta sea crítico.



- El uso del simulador y de Microchip Studio me permitió entender lo que realmente ocurre dentro del microcontrolador a nivel de hardware. Observar cómo mis líneas de código en C se traducen a instrucciones específicas de ensamblador (como sbi, cbi, in, out), y ver cómo cambian los bits en la memoria paso a paso, me dio una perspectiva mucho más profunda sobre cómo optimizar mis programas en lugar de solo escribir código a ciegas.

9. Bibliografía

[1] W. P. Chamba Zaragocín, E. R. Guamán Quinche, F. J. Alvarez Pineda, J. O. Guamán Quinche, G. B. Zaragocín Arrobo y G. Á. Valarezo Benítez, Análisis y diseño preliminar de software basado en el modelado de requisitos y casos de uso. Un enfoque teórico y práctico. Loja, Ecuador: Universidad Nacional de Loja, 2025. Disponible en:
<https://unl.edu.ec/investigacion/produccion-cientifica/analisis-y-diseno-preliminar-de-software-basado-en-el-modelado>